

My GitHub repository: - <https://github.com/Harniii/InvestmentApp>

Software Requirements:

Python: Both the server and client codes are Python-based. Ensure Python version 3.6 or newer is installed on your system.

SQLite: The server-side code utilizes SQLite for database operations. SQLite3 is included in Python's standard library for Python 3.6 and above, so no additional installation is required for SQLite.

SQLite Download Link - <https://www.sqlite.org/download.html>

External Python Libraries: The code utilizes external libraries for various functions, such as web scraping and data visualization. These libraries need to be installed via pip, Python's package manager.

requests: For making HTTP requests to web pages for scraping market data.

Beautiful Soup (bs4): For parsing HTML and XML documents, used in conjunction with requests for web scraping.

matplotlib: For creating static, interactive, and animated visualizations in Python.

pandas: For data manipulation and analysis. Used primarily on the client-side for handling market data.

To simplify the setup process, a requirements.txt file has been added, listing all the external Python libraries required to run the codebase. These libraries include requests, beautifulsoup4, matplotlib, and pandas.

Installing Dependencies:

Before running the server or client applications, install the required dependencies listed in the requirements.txt file using pip:

```
pip install -r requirements.txt
```

This command will automatically download and install the specified versions of the required libraries.

Additional Setup:

Network Configuration: Ensure that the server and client machines (if different) are configured to allow TCP connections over the specified port (default is 8000). Firewall and network settings may need to be adjusted to allow this communication.

Database Files: The server-side code expects SQLite database files (users.db, stocks.csv, and cryptocurrencies.csv) to be accessible. Make sure the server has permission to read from and write to these files.

CSV Files for Market Data: The server-side scrapers (Crypto Scraper and Stock Scraper) write market data to CSV files (cryptocurrencies.csv and stocks.csv, respectively). Ensure that the directory where the server code runs has write permissions to create and update these files.

Environment Compatibility: The client-side application uses Tkinter for the GUI, which should work out of the box on most operating systems. However, on some Linux distributions, you may need to install the Tkinter library separately if it's not included with your Python installation. For macOS, using the TkAgg backend for matplotlib might require additional setup for compatibility.

Running the Code:

Start the Server: Run the server-side Python script first to ensure that the server is listening for incoming client connections.

Launch the Client Application: Run the client-side Python script. The GUI should launch, allowing you to interact with the server through the graphical interface.

By ensuring these requirements and setups are met, you should be able to run the server and client applications successfully for a complete system that allows user registration, authentication, account management, and market data visualization.

Server-Side Application

Database Initialization: Initializes three SQLite databases for users, portfolios, and transactions with appropriate fields and constraints. This includes creating tables if they don't exist and establishing relationships between them.

Server Functions:

Authentication and Registration: Functions to register new users and authenticate existing ones based on their username and password.

Account Operations: Includes depositing funds, withdrawing funds, and checking account balance.

Investment Operations: Functions to handle investment actions such as investing in markets (buying), selling stock, and updating portfolio information.

Database Connection Manager: A context manager for handling database connections safely and efficiently.

Client Handler: A function that handles client connections, receives requests (such as login, register, deposit, etc.), and sends appropriate responses back to the client.

Market Data Scrapers: Classes (Crypto Scraper and Stock Scraper) designed to scrape market data (cryptocurrency and stocks) from specified websites and write this data to CSV files.

Server Initialization: Sets up a TCP socket server listening on a specified port, ready to accept connections from clients. Each client connection is handled on a separate thread.

Client-Side GUI Application

Client Functions:

Connection to Server: Functions to connect to the server and send requests for operations like login, registration, deposit, withdrawal, and balance check.

GUI Windows: Several classes define different windows for the application, including the main login window, registration window, account window, and windows for viewing and managing investments.

Investment and Portfolio Management: Allows the user to view their balance and portfolio, make new investments, and view the performance of their investments. It integrates with matplotlib for plotting graphs.

Graphical User Interface (GUI):

Tkinter: Uses Tkinter for the GUI, with custom widgets like placeholder entries and windows that center themselves on the screen.

Matplotlib Integration: For visualizing investment data, the client application integrates matplotlib plots into Tkinter windows, enabling users to view graphs of market data.

Main Functionality:

The main window allows users to log in or register. Once authenticated, the user can access their account window, where they can perform transactions (deposit, withdraw), check their balance, and manage their investments.

Specialized windows facilitate specific actions, like registering a new user, making an investment, or viewing one's investment portfolio in detail.

Overall, the codebase combines network programming for client-server communication, database management for storing user and transaction data, web scraping for market data collection, and graphical user interface design for user interaction.